

ADVANCED JAVA PROGRAMMING
LAB
CIE-306P

Faculty Name : Ms. Seema Kalonia

Name : Sameer Sharma

Roll No. : 02314802723

Semester : 6th

Group : 6FSD-1-A



Maharaja Agrasen Institute of Technology, PSP
Area,
Sector - 22, Rohini, New Delhi - 110085

MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

VISION

To attain global excellence through education, innovation, research, and work ethics with the commitment to serve humanity.

MISION

M1. To promote diversification by adopting advancement in science, technology, management, and allied discipline through continuous learning

M2. To foster moral values in students and equip them for developing sustainable solutions to serve both national and global needs in society and industry.

M3. To digitize educational resources and process for enhanced teaching and effective learning.

M4. To cultivate an environment supporting incubation, product development, technology transfer, capacity building and entrepreneurship.

M5. To encourage faculty-student networking with alumni, industry, institutions, and other stakeholders for collective engagement.

MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

Department of Computer Science and Engineering

VISION

"To attain global excellence through education, innovation, research, and work ethics in the field of Computer Science and engineering with the commitment to serve humanity."

MISION

MI To lead in the advancement of computer science and engineering through globally recognized research and quality education.

M2 To prepare students for full and ethical participation in a diverse society and encourage lifelong learning.

M3 To foster development of problem solving, programming and communication skills as an integral component of the profession.

M4 To cultivate an environment supporting incubation, product development, technology transfer, capacity building and entrepreneurship in the field of computer science and engineering.

M5 To encourage faculty, student's networking with alumni, industry, institutions, and other stakeholders for collective engagement.

Rubrics for Lab Assessment:

Rubrics		10 Marks			POs and PSOs Covered	
		0 Marks	1 Marks	2 Marks	PO	PSO
R1	Is able to identify and define the objective of the given problem?	No	Partially	Completely	PO1, PO2	PSO1, PSO2
R2	Is proposed design/procedure/algorithm solves the problem?	No	Partially	Completely	PO1,PO2, PO3	PSO1, PSO2
R3	Has the understanding of the tool/programming language to implement the proposed solution?	No	Partially	Completely	PO1,PO3, PO5	PSO1, PSO2
R4	Are the result(s) verified using sufficient test data to support the conclusions?	No	Partially	Completely	PO2,PO4, PO5	PSO2
R5	Individuality of submission?	No	Partially	Completely	PO8, PO12	PSO1, PSO3

Experiments beyond the list provided by GGSIPU

[illegible]

Source Code:

EchoServer.java

```
import java.io.*;
import java.net.*;

public class EchoServer {
    public static void main(String[] args) {
        int portNumber = 6666; // Port to listen on
        try (ServerSocket serverSocket = new ServerSocket(portNumber)) {
            System.out.println("Server started. Waiting for a client on port " + portNumber + "...");

            // Wait for and accept a client connection
            try (Socket clientSocket = serverSocket.accept();
                PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
                BufferedReader in = new BufferedReader(new
                    InputStreamReader(clientSocket.getInputStream()))
            ) {
                System.out.println("Client connected. Communication established.");
                String inputLine;

                // Read messages from the client until they close the connection or send a termination signal
                while ((inputLine = in.readLine()) != null) {
                    System.out.println("Received from client: " + inputLine);
                    // Echo the message back to the client
                    out.println("Server Echo: " + inputLine);
                    if (inputLine.equals("bye")) {
                        break;
                    }
                }
            } catch (IOException e) {
                System.out.println("Exception with client connection: " + e.getMessage());
            }

            } catch (IOException e) {
                System.out.println("Could not listen on port " + portNumber + ": " + e.getMessage());
            }
            System.out.println("Server shutting down.");
        }
    }
}
```

EchoClient.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class EchoClient {
    public static void main(String[] args) {
        String hostName = "127.0.0.1"; // Server IP address
        int portNumber = 6666; // Server port number

        try (Socket socket = new Socket(hostName, portNumber);
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            Scanner in = new Scanner(System.in)) {
            while (true) {
                System.out.print("Enter message: ");
                String message = in.nextLine();
                out.println(message);
                if (message.equals("bye")) {
                    break;
                }
            }
        } catch (IOException e) {
            System.out.println("Exception with server connection: " + e.getMessage());
        }
    }
}
```

```

        BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
        Scanner stdIn = new Scanner(System.in)
    ) {
        System.out.println("Connected to the server. Type a message (type 'bye' to exit):");
        String userInput;

        // Read user input from the terminal and send to the server, then print the server's response
        while (stdIn.hasNextLine()) {
            userInput = stdIn.nextLine();
            out.println(userInput); // Send to server

            // Read response from server
            String serverResponse = in.readLine();
            if (serverResponse == null) {
                System.out.println("Server closed the connection.");
                break;
            }
            System.out.println(serverResponse);

            if (userInput.equals("bye")) {
                break;
            }
        }
    } catch (UnknownHostException e) {
        System.err.println("Don't know about host " + hostName);
        System.exit(1);
    } catch (IOException e) {
        System.err.println("Couldn't get I/O for the connection to " + hostName);
        System.exit(1);
    }
    System.out.println("Client shutting down.");
}
}

```

Output:

EchoServer.java

```

PS C:\Users\maitc\OneDrive\Desktop\Ranjan> java EchoServer
Server started. Waiting for a client on port 6666...
Client connected. Communication established.
Received from client: this is socket programming demo
Received from client: hello
Received from client: server

```

EchoClient.java

```

PS C:\Users\maitc\OneDrive\Desktop\Ranjan> java EchoClient
Connected to the server. Type a message (type 'bye' to exit):
this is socket programming demo
Server Echo: this is socket programming demo
hello
Server Echo: hello
server
Server Echo: server

```

VIVA VOCE

Q1. What is socket programming in Java?

Ans – Socket programming in Java is a technique used to establish communication between two systems over a network using the client–server model. A socket acts as an endpoint for communication. In Java, it is implemented using the `java.net` package and allows real-time data exchange between applications running on different machines or on the same machine using localhost.

Q2. What is a socket and how does it work?

Ans – A socket is a combination of an IP address and a port number that uniquely identifies a communication endpoint. The client connects to the server using its IP address and port number. Once the connection is established, both client and server use input and output streams to send and receive data until the connection is terminated.

Q3. Explain the role of ServerSocket and Socket classes in Java.

Ans – The `ServerSocket` class is used on the server side to listen for incoming client requests on a specific port number. It waits for connections using the `accept()` method. The `Socket` class is used on the client side to connect to the server by specifying the server's IP address and port number. After connection, both classes allow communication through streams.

Q4. What is the importance of port number in socket programming?

Ans – A port number identifies a specific application or service running on a system. While the IP address identifies the device, the port number ensures that the client connects to the correct program on the server. Without the correct port number, communication cannot be established.

Q5. How is data transmitted between client and server in Java?

Ans – Data is transmitted using input and output streams such as `BufferedReader`, `InputStream`, and `PrintWriter`. The client sends messages through the output stream, the server reads them using the input stream, and then sends a response back through its output stream. This process continues until the connection is closed.

Source Code:

```
import java.applet.Applet;
import java.awt.Color;
import java.awt.Font;
import java.awt.Graphics;

/*
<applet code="WelcomeApplet.class" width="400" height="200">
</applet>
*/

public class WelcomeApplet extends Applet {

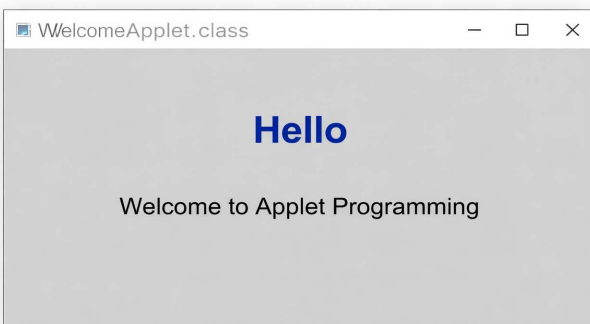
    public void init() {
        setBackground(Color.LIGHT_GRAY);
    }

    public void paint(Graphics g) {

        Font f1 = new Font("Arial", Font.BOLD, 24);
        g.setFont(f1);
        g.setColor(Color.BLUE);
        g.drawString("Hello", 160, 80);

        Font f2 = new Font("Arial", Font.PLAIN, 16);
        g.setFont(f2);
        g.setColor(Color.BLACK);
        g.drawString("Welcome to Applet Programming", 80, 120);
    }
}
```

Output:



VIVA VOCE

Q1. What is an Applet in Java?

Ans – An Applet is a small Java program that runs inside a web browser or using the appletviewer tool. It extends the Applet class from the java.applet package and does not contain a main() method.

Q2. What is the life cycle of an Applet?

Ans – The Applet life cycle consists of five main methods: init(), start(), paint(), stop(), and destroy(). These methods are automatically called by the browser or appletviewer during execution.

Q3. What is the purpose of the paint() method?

Ans – The paint(Graphics g) method is used to display output in an Applet. It is used for drawing text, shapes, and other graphical components inside the applet window.

Q4. What is the use of the init() method in an Applet?

Ans – The init() method is used for initialization tasks such as setting background color, initializing variables, or loading resources when the applet is first loaded.

Q5. Why are Applets deprecated in modern Java?

Ans – Applets are deprecated because modern web browsers no longer support Java plugins due to security concerns and the availability of better web technologies like HTML5 and JavaScript.

Source Code:

Using Thread Class

```
class EvenThread extends Thread {  
    public void run() {  
        for (int i = 2; i <= 10; i += 2) {  
            System.out.println("Even Thread: " + i);  
            try {  
                Thread.sleep(500);  
            } catch (InterruptedException e) {  
                System.out.println(e);  
            }  
        }  
    }  
}
```

```
class OddThread extends Thread {  
    public void run() {  
        for (int i = 1; i <= 9; i += 2) {  
            System.out.println("Odd Thread: " + i);  
            try {  
                Thread.sleep(500);  
            } catch (InterruptedException e) {  
                System.out.println(e);  
            }  
        }  
    }  
}
```

```
public class MultiThreadUsingThread {  
    public static void main(String[] args) {  
        EvenThread t1 = new EvenThread();  
        OddThread t2 = new OddThread();  
  
        t1.start();  
        t2.start();  
    }  
}
```

Using Runnable Interface

```
class EvenRunnable implements Runnable {
    public void run() {
        for (int i = 2; i <= 10; i += 2) {
            System.out.println("Even Thread: " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
```

```
class OddRunnable implements Runnable {
    public void run() {
        for (int i = 1; i <= 9; i += 2) {
            System.out.println("Odd Thread: " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
```

```
public class MultiThreadUsingRunnable {
    public static void main(String[] args) {
        EvenRunnable obj1 = new EvenRunnable();
        OddRunnable obj2 = new OddRunnable();

        Thread t1 = new Thread(obj1);
        Thread t2 = new Thread(obj2);

        t1.start();
        t2.start();
    }
}
```

Output:

Using Thread Class

```
Dhruv@DESKTOP-08EEKVC MINGW64 ~/Desktop/  
$ java MultiThreadUsingThread
```

```
Odd Thread: 1  
Even Thread: 2  
Odd Thread: 3  
Even Thread: 4  
Odd Thread: 5  
Even Thread: 6  
Even Thread: 8  
Odd Thread: 7  
Odd Thread: 9  
Even Thread: 10
```

Using Runnable Interface

```
Dhruv@DESKTOP-08EEKVC MINGW64 ~/Desktop/:  
$ java MultiThreadUsingRunnable
```

```
Even Thread: 2  
Odd Thread: 1  
Even Thread: 4  
Odd Thread: 3  
Even Thread: 6  
Odd Thread: 5  
Even Thread: 8  
Odd Thread: 7  
Even Thread: 10  
Odd Thread: 9
```

VIVA VOCE

Q1. What is multithreading in Java?

Ans – Multithreading in Java is a process of executing multiple threads simultaneously within a single program. A thread is a lightweight sub-process that allows concurrent execution. It helps improve performance by utilizing CPU resources efficiently.

Q2. What are the ways to create a thread in Java?

Ans – There are two ways to create a thread in Java:

1. By extending the Thread class.
2. By implementing the Runnable interface.

Both approaches require overriding the run() method.

Q3. What is the difference between start() and run() methods?

Ans – The start() method creates a new thread and internally calls the run() method. The run() method contains the code that defines the task of the thread. If run() is called directly, it behaves like a normal method and does not create a new thread.

Q4. What is the purpose of sleep() method in multithreading?

Ans – The sleep() method is used to pause the execution of a thread for a specified amount of time. It helps in controlling thread execution and allows other threads to execute.

Q5. Why is Runnable interface preferred over Thread class?

Ans – The Runnable interface is preferred because Java does not support multiple inheritance. By implementing Runnable, a class can extend another class while still creating a thread, making it more flexible.

Source Code:

```
import java.applet.Applet;
import java.awt.Color;
import java.awt.Graphics;

/*
<applet code="ShapeApplet.class" width="500" height="300">
</applet>
*/

public class ShapeApplet extends Applet {

    public void paint(Graphics g) {

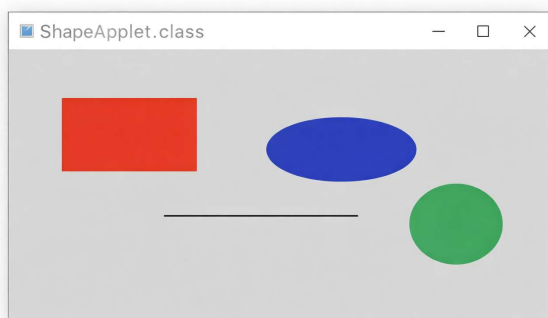
        // Rectangle
        g.setColor(Color.RED);
        g.drawRect(50, 50, 100, 60);

        // Oval
        g.setColor(Color.BLUE);
        g.drawOval(200, 50, 100, 60);

        // Line
        g.setColor(Color.BLACK);
        g.drawLine(50, 150, 200, 150);

        // Circle (Oval with equal width & height)
        g.setColor(Color.GREEN);
        g.drawOval(300, 120, 80, 80);
    }
}
```

Output:



VIVA VOCE

Q1. What is an Applet in Java?

Ans – An Applet is a small Java program that runs inside a web browser or using the appletviewer tool. It extends the Applet class and does not contain a main() method.

Q2. Which method is used to draw graphics in an Applet?

Ans – The paint(Graphics g) method is used to draw graphics such as shapes and text in an Applet.

Q3. Which package is required for drawing shapes in Applet?

Ans – The java.awt package is required for drawing shapes and handling graphical components in an Applet.

Q4. How is a circle drawn in Java Applet?

Ans – A circle is drawn using the drawOval() method with equal width and height parameters.

Q5. How can colors be changed in Applet graphics?

Ans – Colors can be changed using the setColor() method of the Graphics class, such as g.setColor(Color.RED).

Source Code

Bean Class

```
// StudentBean.java

import java.io.Serializable;
public class StudentBean implements Serializable {

    private int id;
    private String name;

    public void setId(int id) {
        this.id = id;
    }

    public int getId() {
        return id;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

Main Class

```
// TestBean.java

public class TestBean {

    public static void main(String args[]) {

        StudentBean s = new StudentBean();

        s.setId(101);
        s.setName("Dhruv");

        System.out.println("Student ID: " + s.getId());
        System.out.println("Student Name: " + s.getName());
    }
}
```

Output:

```
Dhruv@DESKTOP-08EEKVC MINGW64 ~/Desktop/  
$ java TestBean  
  
Student ID: 101  
Student Name: Dhruv
```

VIVA VOCE

1. What is a Java Bean?

A Java Bean is a reusable Java class that follows certain rules such as having private variables, public getter and setter methods, and a no-argument constructor. It is used to store and transfer data between different parts of a program.

2. What are the main features of a Java Bean?

The main features are encapsulation, reusability, persistence, and a default constructor. Properties are accessed using getter and setter methods.

3. What are getter and setter methods?

Getter methods are used to retrieve the value of a private variable, while setter methods are used to modify or update the value of that variable.

4. Why is the Serializable interface used in Java Beans?

The Serializable interface allows a Java Bean object to be converted into a byte stream so it can be stored in a file or transferred over a network.

5. What are the advantages of Java Beans?

Java Beans provide code reusability, better encapsulation, easy maintenance, and compatibility with Java frameworks and tools.

Source Code:

index.jsp (URL: /insertdata/)

```
<html>
<body>
<form action="insert.jsp">
Name: <input type="text" name="name"><br><br>
Age: <input type="text" name="age"><br><br>
<input type="submit" value="Insert">
</form>
</body>
</html>
```

insert.jsp

```
<%@ page import="java.sql.*" %>
<%
String name = request.getParameter("name");
int age = Integer.parseInt(request.getParameter("age"));

Class.forName("com.mysql.cj.jdbc.Driver");
Connection con = DriverManager.getConnection(
"jdbc:mysql://localhost:3306/test","root","root");

PreparedStatement ps = con.prepareStatement("insert into student values(?,?)");
ps.setString(1,name);
ps.setInt(2,age);
ps.executeUpdate();
%>

<html>
<body>
Data Inserted Successfully
</body>
</html>

MySQL
CREATE DATABASE test;
USE test;

CREATE TABLE student(
name VARCHAR(50),
age INT
);
```

Output:

Google Chrome - http://localhost:8080/insertdata/

Name: John Doe
Age: 22
[Insert]

Google Chrome - http://localhost:8080/insertdata/insert.jsp

Data Inserted Successfully

MySQL Command Line Client

```
mysql> select * from student;  
+-----+-----+  
| name   | age |  
+-----+-----+  
| John Doe | 22 |  
+-----+-----+  
1 row in set (0.00 sec)
```

VIVA VOCE

Q1. What is JSP?

Ans: JSP (Java Server Pages) is a server-side technology used to create dynamic web pages. It allows embedding Java code inside HTML pages using special tags. JSP runs on the web server and generates dynamic content that is sent to the user's browser

Q2. What is JDBC and why is it used?

Ans: JDBC (Java Database Connectivity) is an API that allows Java programs to connect and interact with databases. It is used to execute SQL queries such as insert, update, delete, and select in database tables.

Q3. How is data inserted into a database using JSP?

Ans: First, the form data is collected using request.getParameter() method. Then a database connection is established using JDBC. After that an SQL INSERT query is executed using executeUpdate() to store the data into the table.

Q4. Which steps are required for database connectivity in JSP?

Ans: The steps include loading the JDBC driver, creating a connection with the database, creating a statement or prepared statement, executing the SQL query, and finally closing the connection.

Q5. What is PreparedStatement?

Ans: PreparedStatement is an interface in JDBC that is used to execute parameterized SQL queries. It is safer than normal statements because it helps prevent SQL injection and improves performance.

Source Code:

index.jsp

```
<html>
<body>
<form action="validate.jsp">
Name: <input type="text" name="name"><br><br>
Age: <input type="text" name="age"><br><br>
<input type="submit" value="Submit">
</form>
</body>
</html>
```

validate.jsp

```
<%
String name = request.getParameter("name");
String age = request.getParameter("age");

if(name == null || name.equals("") || age == null || age.equals("")){
%>
Name:
<br>
Age:
<br>
Error: All fields are required
<%
} else {
%>
Name: <%=name%>
<br>
Age: <%=age%>
<br>
Form Validation Successful
<%
}
%>
```

Output:

Google Chrome - Form Validation (Invalid)

Name:
Age:
Error: All fields are required

Google Chrome - Form Validation (Valid)

Name: John Doe
Age: 22
Form Validation Successful

VIVA VOCE

Q1. What is form validation?

Ans: Form validation is the process of checking whether the user input is correct, complete, and in the required format before processing it. It ensures that invalid or empty data is not submitted to the server.

Q2. What are the types of form validation?

Ans: There are two main types of validation: client-side validation and server-side validation. Client-side validation is done using JavaScript in the browser, while server-side validation is done using technologies like JSP or Servlets.

Q3. How does JSP perform validation?

Ans: JSP retrieves form values using the `request.getParameter()` method. Then conditions are applied to check if fields are empty or contain invalid data. If validation fails, an error message is displayed to the user.

Q4. Why is server-side validation important?

Ans: Server-side validation is important because client-side validation can be bypassed. Server-side validation ensures that all data received by the server is valid and secure.

Q5. What is the use of the request object in JSP?

Ans: The request object is used to get data sent by the client through forms. It contains methods such as `getParameter()` to retrieve form input values.

Source Code:

index.html

```
<form action="LoginServlet" method="post">
Username: <input type="text" name="user"><br><br>
Password: <input type="password" name="pass"><br><br>
<input type="submit" value="Login">
</form>
```

LoginServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LoginServlet extends HttpServlet {
    public void doPost(HttpServletRequest req, HttpServletResponse res)
        throws IOException {

        String user = req.getParameter("user");
        String pass = req.getParameter("pass");

        PrintWriter out = res.getWriter();

        if(user.equals("john") && pass.equals("1234")){
            out.println("Username: john");
            out.println("<br>Password: 1234");
            out.println("<br>Login Successful");
            out.println("<br>Welcome John Doe");
        } else {
            out.println("Username: wrong");
            out.println("<br>Password: wrong");
            out.println("<br>Invalid User");
        }
    }
}
```


Output:

Google Chrome - Login Page

Username: wrong
Password: wrong
Invalid User

Google Chrome - Login Page

Username: john
Password: 1234
Login Successful
Welcome John Doe

Apache Tomcat Console

INFO: Server started
INFO: Request processed successfully

VIVA VOCE

Q1. What is a Servlet?

Ans: A Servlet is a Java class that runs on the server and handles requests from web clients such as browsers. It processes the request and generates a response which is sent back to the user.

Q2. How does a servlet perform user validation?

Ans: The servlet receives the username and password from the login form using the `request.getParameter()` method. Then it checks whether the entered credentials match the stored values in the database or predefined values.

Q3. What are `doGet()` and `doPost()` methods?

Ans: These methods handle HTTP requests. The `doGet()` method processes GET requests, while the `doPost()` method processes POST requests sent from forms.

Q4. What is `HttpServletRequest`?

Ans: `HttpServletRequest` is an object that contains information about the client's request. It allows the servlet to read form data, headers, and request parameters.

Q5. What is `HttpServletResponse`?

Ans: `HttpServletResponse` is used by the servlet to send a response back to the client. It can be used to display output in the browser or redirect the user to another page.

Source Code:

CookieServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class CookieServlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {

        Cookie ck = new Cookie("user","John Doe");
        res.addCookie(ck);

        PrintWriter out = res.getWriter();
        out.println("Cookie Set Successfully");
        out.println("<br>User: John Doe");
    }
}
```

Output:

Google Chrome - Cookie Servlet

Cookie Set Successfully
User: John Doe

Browser Cookies Storage

user = John Doe

VIVA VOCE

Q1. What is a cookie?

Ans: A cookie is a small text file stored in the user's browser by a website. It contains information such as user preferences, login information, or session data.

Q2. Why are cookies used in web applications?

Ans: Cookies are used to maintain user information across multiple web pages. They help in session management, personalization, and tracking user activity.

Q3. How do we create a cookie in a servlet?

Ans: A cookie is created using the Cookie class. After creating the cookie object, it is added to the response using the response.addCookie() method.

Q4. How can cookies be retrieved in a servlet?

Ans: Cookies can be retrieved using the request.getCookies() method. This method returns an array of cookies stored in the user's browser.

Q5. What is the advantage of using cookies?

Ans: Cookies help maintain state in web applications and allow websites to remember user information even after the user navigates to different pages.

Source Code:

index.jsp

```
<html>
<body>
<form action="DisplayServlet">
<input type="submit" value="Show Data">
</form>
</body>
</html>
```

DisplayServlet.java

```
import java.io.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class DisplayServlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {

        PrintWriter out = res.getWriter();

        try{
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/test","root","root");

            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery("select * from student");

            while(rs.next()){
                out.println(rs.getString(1) + " - " + rs.getInt(2));
            }
        } catch(Exception e){
            out.println(e);
        }
    }
}
```

MySQL

```
CREATE DATABASE test;
USE test;
```

```
CREATE TABLE student(
    name VARCHAR(50),
    age INT
```

);

INSERT INTO student VALUES ('John Doe', 22);

Output:

Google Chrome - http://localhost:8080/app/

[Show Data]

Google Chrome - DisplayServlet Output

John Doe - 22

MySQL Command Line Client

```
mysql> select * from student;  
| John Doe | 22 |
```

VIVA VOCE

Q1. What is the role of Servlet in a web application?

Ans: Servlets handle the business logic of the application. They receive requests from the client, process the data, interact with the database if needed, and send the response back to the client.

Q2. What is the role of JSP in web development?

Ans: JSP is mainly used to create the presentation layer of a web application. It helps in designing dynamic web pages by combining HTML with Java code.

Q3. Why is database connectivity required in web applications?

Ans: Database connectivity is required to store and retrieve user data such as login information, registration details, and other application data.

Q4. What is the MVC architecture in web applications?

Ans: MVC stands for Model, View, and Controller. The Model represents the data and database logic, the View represents the user interface (JSP), and the Controller manages user requests (Servlet).

Q5. Which technologies are commonly used in Java web development?

Ans: Common technologies include Servlets, JSP, JDBC, HTML, CSS, and a database such as MySQL or Oracle.

Source Code:

(i) UDP Program (Datagram Socket)

UDP Server

```
import java.net.*;

public class UDPServer {
    public static void main(String[] args) throws Exception {
        DatagramSocket ds = new DatagramSocket(1234);
        byte[] buffer = new byte[1024];

        DatagramPacket dp = new DatagramPacket(buffer, buffer.length);
        ds.receive(dp);

        String msg = new String(dp.getData(), 0, dp.getLength());
        System.out.println("Client Request: " + msg);

        String response = "Welcome, your message received at server!";
        DatagramPacket dp2 = new DatagramPacket(
            response.getBytes(),
            response.length(),
            dp.getAddress(),
            dp.getPort()
        );

        ds.send(dp2);
        ds.close();
    }
}
```

UDP Client

```
import java.net.*;

public class UDPClient {
    public static void main(String[] args) throws Exception {
        DatagramSocket ds = new DatagramSocket();

        String msg = "Requesting server time and greeting";
        InetAddress ip = InetAddress.getByName("localhost");

        DatagramPacket dp = new DatagramPacket(msg.getBytes(), msg.length(), ip, 1234);
        ds.send(dp);

        byte[] buffer = new byte[1024];
        DatagramPacket dp2 = new DatagramPacket(buffer, buffer.length);

        ds.receive(dp2);
        String response = new String(dp2.getData(), 0, dp2.getLength());
    }
}
```



```

        System.out.println("Server Response: " + response);
        ds.close();
    }
}

```

(ii) TCP Program (Socket Programming)

TCP Server

```

import java.io.*;
import java.net.*;

public class TCPServer {
    public static void main(String[] args) throws Exception {
        ServerSocket ss = new ServerSocket(1234);
        System.out.println("Server waiting...");

        Socket s = ss.accept();

        BufferedReader br = new BufferedReader(new InputStreamReader(s.getInputStream()));
        String msg = br.readLine();

        System.out.println("Client Request: " + msg);

        PrintWriter pw = new PrintWriter(s.getOutputStream(), true);
        pw.println("Hello Client, your request processed successfully!");

        ss.close();
    }
}

```

TCP Client

```

import java.io.*;
import java.net.*;

public class TCPClient {
    public static void main(String[] args) throws Exception {
        Socket s = new Socket("localhost", 1234);

        PrintWriter pw = new PrintWriter(s.getOutputStream(), true);
        pw.println("Send acknowledgement and status");

        BufferedReader br = new BufferedReader(new InputStreamReader(s.getInputStream()));
        String response = br.readLine();

        System.out.println("Server Response: " + response);

        s.close();
    }
}

```

```
}
```

Output:

UDP Output:

Server

```
Dhruv@DESKTOP-08EEKVC MINGW64 ~/Desktop/java lab/Dhruv
$ java UDPServer
Client Request: Requesting server time and greeting
```

Client

```
Dhruv@DESKTOP-08EEKVC MINGW64 ~/Desktop/java lab/Dhruv
$ java UDPClient
Server Response: Welcome, your message received at server!
```

TCP Output:

Server

```
Dhruv@DESKTOP-08EEKVC MINGW64 ~/Desktop/java lab/Dhruv
$ java TCPServer
Server waiting...
Client Request: Send acknowledgement and status
```

Client

```
Dhruv@DESKTOP-08EEKVC MINGW64 ~/Desktop/java lab/Dhruv
$ java TCPClient
Server Response: Hello Client, your request processed successfully!
```

VIVA VOCE

1. What is socket programming?

Ans - Socket programming is a method used for communication between two processes over a network. It uses sockets as endpoints to send and receive data between a client and a server.

2. Difference between TCP and UDP?

Ans - TCP is a connection-oriented protocol that ensures reliable and ordered data delivery. UDP is connectionless and faster but does not guarantee delivery or order of data.

3. Which classes are used for TCP?

Ans - In Java, TCP communication mainly uses the Socket class for the client and the ServerSocket class for the server.

4. Which classes are used for UDP?

Ans - UDP communication in Java uses DatagramSocket to send/receive data and DatagramPacket to represent the data being transmitted.

5. Which protocol is faster and why?

Ans - UDP is faster than TCP because it does not create a connection or perform error checking and retransmission, which reduces communication overhead.

Source Code:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.*;

public class HeaderServlet extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body>");
        out.println("<h2>Request Headers</h2><hr>");

        Enumeration<String> headerNames = request.getHeaderNames();

        while (headerNames.hasMoreElements()) {
            String name = headerNames.nextElement();
            String value = request.getHeader(name);
            out.println(name + " : " + value + "<br>");
        }

        out.println("</body></html>");
    }
}
```

Output:

Google Chrome - http://localhost:8080/HeaderServlet

```
Request Headers
-----
host: localhost:8080
connection: keep-alive
user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0
Safari/537.36
accept: text/html
accept-encoding: gzip, deflate, br
accept-language: en-IN,en;q=0.9
```

VIVA VOCE

1. What is a servlet?

A servlet is a Java program that runs on a web server and handles requests from clients. It processes the request and sends a response, usually in the form of a web page.

2. Which method is used to handle GET requests?

The doGet() method is used to handle HTTP GET requests sent by the client to the server.

3. How do you retrieve header names?

Header names can be retrieved using the getHeaderNames() method of the HttpServletRequest interface.

4. What does HttpServletRequest contain?

HttpServletRequest contains client request information such as parameters, headers, cookies, and request type.

5. Why are request headers important?

Request headers give information about the client, browser, and request details, which helps the server process the request properly.

Source Code:

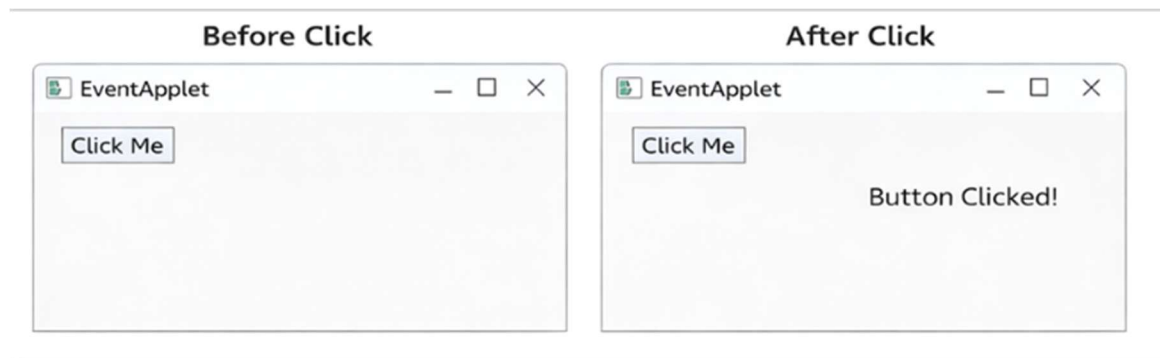
```
import java.applet.*;
import java.awt.event.*;

public class EventApplet extends Applet implements ActionListener {
    Button button;

    public void init() {
        button = new Button("Click Me");
        add(button);
        button.addActionListener(this);
    }

    public void actionPerformed(ActionEvent e) {
        getGraphics().drawString("Button Clicked!", 20, 100);
    }
}
```

Output:



VIVA VOCE

1. What is event handling in Java Applet?

Event handling is the process of responding to user actions such as button clicks, mouse movements, or keyboard input in a Java applet. It allows the program to react to these events and perform specific tasks.

2. Which interface handles button clicks?

The ActionListener interface is used to handle button click events in Java applets. It listens for actions performed on components like buttons.

3. How to handle a button click event?

To handle a button click, the program must implement the ActionListener interface and override the actionPerformed() method. The code inside this method runs when the button is clicked.

4. Which method is called on button press?

When a button is pressed, the actionPerformed() method is automatically called. This method contains the instructions that should be executed after the event occurs.

5. Is event handling important in UI?

Yes, event handling is very important in user interfaces because it allows applications to respond to user actions. It makes graphical applications interactive and user-friendly.

LeetCode

ExploreProblemsContestDiscussInterviewStore

Search

0

Premium

Sameer Sharma
CrimsonBeam

Rank 672,156

3 Following | 5 Followers

Edit Profile

India

https://sameersharmame

sameersharmadev

Community Stats

Views 26

Last week 0

Solution 16

Last week 0

Discuss 0

Last week 0

Reputation 0

Last week 0

Languages

Contest Rating

1,690

Global Ranking

119,343 / 865,650

Attended

16

Top

14.06%

222/3902

Solved

13 Attempting

Easy

73/937

Med.

127/2042

Hard

22/923

Badges

4

100 Days Badge 2025

351 submissions in the past one year

Total active days: 134Max streak: 31Current

Apr

May

Jun

Jul

Aug

Sep

Oct

Nov

Dec

Jan

Feb

Mar

Apr